

Tecnatura Superior en Desarrollo de Software

Práctica 04 — Capa Application: Casos de Uso, Puertos y DTOs

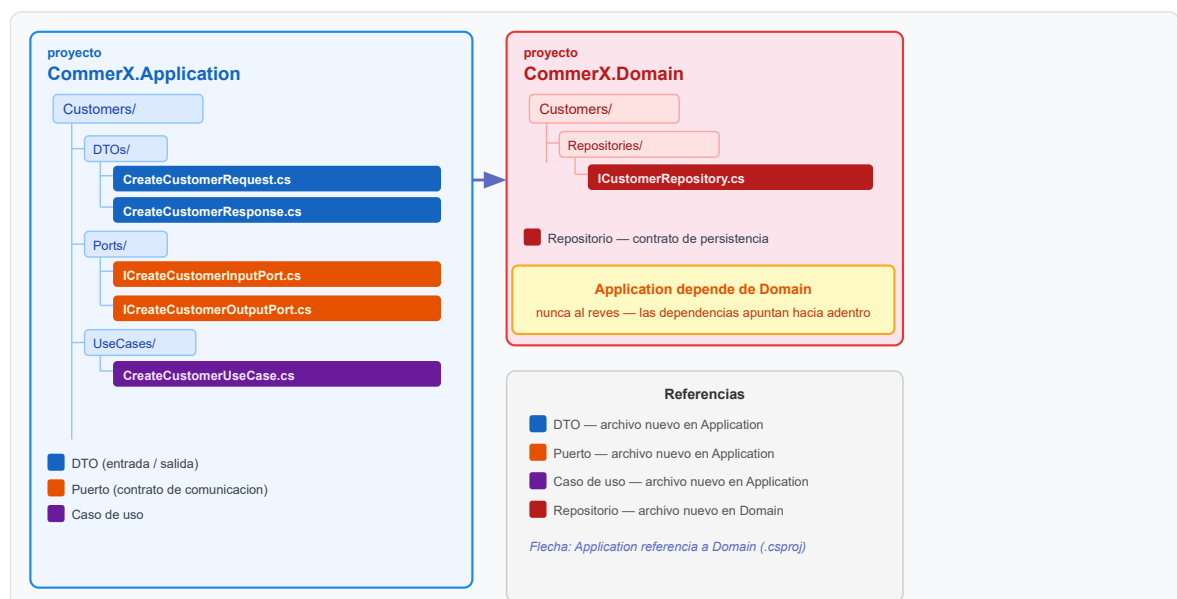
Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

Ejemplo práctico

El caso de uso **CU-CLI-001 — Alta de Cliente** permite implementar de forma completa todos los elementos de la capa *Application*: DTOs de entrada y salida, puertos de comunicación, interfaz de repositorio y el caso de uso concreto.

Objetivo: Crear los DTOs `CreateCustomerRequest` y `CreateCustomerResponse`, definir los puertos `ICreateCustomerInputPort` e `ICreateCustomerOutputPort`, declarar `ICustomerRepository` en Domain e implementar `CreateCustomerUseCase` en Application. La solución debe compilar sin errores y respetar la Regla de Dependencia: *Application conoce Domain — nunca al revés*.

Archivos a crear — estructura de proyectos



Archivo 1 — CommerX.Domain / Customers / Repositories / ICustomerRepository.cs

¿Por qué en Domain y no en Application?

El repositorio opera con la entidad `Customer`, que es un tipo del dominio. Si la interfaz viviera en Application, el Dominio debería referenciar a Application para conocer sus propias entidades — rompiendo la Regla de Dependencia. Al ubicarla en Domain, Application puede usarla sin crear dependencias circulares.

```
1 // CommerX.Domain/Customers/Repositories/ICustomerRepository.cs
2 // contrato de persistencia – Infrastructure lo implementará con EF Core
3 using CommerX.Domain.Customers.Entities;
4
5 namespace CommerX.Domain.Customers.Repositories;
6
7 public interface ICustomerRepository
8 {
9     // verifica si ya existe un cliente con ese número de documento
10    // retorna bool – el UseCase solo necesita saber si existe, no obtener la entidad
11    Task<bool> ExistsByDocumentAsync(string document);
12
13    // verifica si ya existe un cliente registrado con ese email
14    Task<bool> ExistsByEmailAsync(string email);
15
16    // persiste el nuevo cliente – SaveChanges lo confirmará UnitOfWork (próximo módulo)
17    Task AddAsync(Customer customer);
18 }
```

Solo se declaran los métodos que este caso de uso necesita — principio ISP. No hay `GetAll()`, `Update()` ni `Delete()` porque CU-CLI-001 no los requiere.

Archivo 2 — CommerX.Application / Customers / DTOs / CreateCustomerRequest.cs

¿Qué hace esta clase?

Transporta los datos que llegan desde la interfaz de usuario hacia el caso de uso. Es inmutable y sin lógica. Usa tipos primitivos de C# — nunca Value Objects del dominio — para no crear dependencias circulares entre capas.

```
1 // CommerX.Application/Customers/DTOs/CreateCustomerRequest.cs
2 namespace CommerX.Application.Customers.DTOs;
```

```
3
4 // class – no record: los DTOs son clases, no tipos de valor
5 public class CreateCustomerRequest
6 {
7     // required: el compilador obliga a inicializar la propiedad
8     // init: solo puede asignarse al construir el objeto – luego es inmutable
9     public required string  FirstName { get; init; }
10    public required string  LastName { get; init; }
11    public required string  Document { get; init; }
12    public required string  Email    { get; init; }
13    public required string  Phone    { get; init; }
14    public required string  Address  { get; init; }
15    // DateOnly: tipo primitivo de C# – no el Value Object BirthDate del dominio
16    public required DateOnly BirthDate { get; init; }
17 }
```

Archivo 3 — CommerX.Application / Customers / DTOs / CreateCustomerResponse.cs

¿Qué hace esta clase?

Devuelve al llamador una confirmación mínima tras el alta exitosa. Nunca expone la entidad `Customer` directamente — solo los datos que la capa externa necesita para confirmar la operación.

```
1 // CommerX.Application/Customers/DTOs/CreateCustomerResponse.cs
2 namespace CommerX.Application.Customers.DTOs;
3
4 public class CreateCustomerResponse
5 {
6     // identificador único generado por el sistema (GUID)
7     public required Guid  CustomerId { get; init; }
8     // datos mínimos para confirmar qué cliente fue registrado
9     public required string FirstName { get; init; }
10    public required string LastName  { get; init; }
11 }
```

Archivo 4 — CommerX.Application / Customers / Ports / ICreateCustomerInputPort.cs

¿Qué hace esta interfaz?

Define el contrato que el caso de uso debe cumplir. La capa InterfaceAdapter depende de este puerto para disparar el caso de uso — nunca del caso de uso concreto. Si mañana se reemplaza `CreateCustomerUseCase` por otra implementación, el llamador no necesita cambiar.

```
1 // CommerX.Application/Customers/Ports/ICreateCustomerInputPort.cs
2 using CommerX.Application.Customers.DTOs;
3
4 namespace CommerX.Application.Customers.Ports;
5
6 // puerto de entrada: recibe la solicitud del mundo exterior
7 public interface ICreateCustomerInputPort
8 {
9     // ejecuta el caso de uso CU-CLI-001 de forma asincrónica
10    Task ExecuteAsync(CreateCustomerRequest request);
11 }
```

Archivo 5 — CommerX.Application / Customers / Ports / ICreateCustomerOutputPort.cs

¿Qué hace esta interfaz?

Permite que el caso de uso entregue el resultado sin acoplarse a ningún mecanismo de presentación. Cada método representa *un resultado posible del caso de uso*. La capa InterfaceAdapter implementa este puerto y decide cómo reaccionar ante cada resultado — el UseCase no sabe nada de eso.

```
1 // CommerX.Application/Customers/Ports/ICreateCustomerOutputPort.cs
2 using CommerX.Application.Customers.DTOs;
3
4 namespace CommerX.Application.Customers.Ports;
5
6 // puerto de salida: un método por cada resultado posible del caso de uso
7 // Task: permite que la implementación concreta realice operaciones asincrónicas
8 public interface ICreateCustomerOutputPort
9 {
10    // RESULTADO 1 – el cliente fue creado exitosamente
11    Task HandleSuccessAsync(CreateCustomerResponse response);
12
13    // RESULTADO 2 – ya existe un cliente con ese número de documento
```

```
14     Task HandleDuplicateDocumentAsync(string document);
15
16     // RESULTADO 3 – ya existe un cliente registrado con ese email
17     Task HandleDuplicateEmailAsync(string email);
18
19     // RESULTADO 4 – una regla de dominio fue violada (DomainException capturada)
20     Task HandleValidationErrorAsync(string message);
21 }
```

¿Por qué métodos separados y no un único Result?

Cada método describe con precisión qué ocurrió. Quien implemente este puerto (en el Módulo 07 vas a conocer a esa clase como el Presenter) sabe exactamente qué mensaje mostrar, qué campo resaltar o a qué pantalla redirigir — sin necesidad de inspeccionar un objeto genérico. El código del receptor expresa la intención con claridad.

Archivo 6 — CommerX.Application / Customers / UseCases / CreateCustomerUseCase.cs

¿Qué hace esta clase?

Implementa `ICreateCustomerInputPort` y orquesta la lógica de aplicación: verifica unicidad de documento y email, delega la creación al dominio y notifica el resultado a través del puerto de salida. No contiene lógica de dominio — esa responsabilidad pertenece a `Customer` y sus Value Objects.

```
1  // CommerX.Application/Customers/UseCases/CreateCustomerUseCase.cs
2  using CommerX.Application.Customers.DTOs;
3  using CommerX.Application.Customers.Ports;
4  using CommerX.Domain.Common.Exceptions;
5  using CommerX.Domain.Customers.Entities;
6  using CommerX.Domain.Customers.Repositories;
7
8  namespace CommerX.Application.Customers.UseCases;
9
10 // sealed: su comportamiento es definitivo – no puede heredarse
11 public sealed class CreateCustomerUseCase : ICreateCustomerInputPort
12 {
13     // contrato de persistencia definido en Domain – Infrastructure lo implementa
14     private readonly ICustomerRepository _repository;
15     // puerto de salida – notifica el resultado a quien corresponda
16     private readonly ICreateCustomerOutputPort _outputPort;
17
18     // las dependencias se inyectan por constructor – Principio DIP
19     public CreateCustomerUseCase(
```

```
20         ICustomerRepository repository,
21         ICreateCustomerOutputPort outputPort)
22     {
23         _repository = repository;
24         _outputPort = outputPort;
25     }
26
27     public async Task ExecuteAsync(CreateCustomerRequest request)
28     {
29         // PASO 1 – Unicidad de documento: regla de negocio de la capa Application
30         if (await _repository.ExistsByDocumentAsync(request.Document))
31         {
32             // notificamos y salimos – no seguimos procesando
33             await _outputPort.HandleDuplicateDocumentAsync(request.Document);
34             return;
35         }
36
37         // PASO 2 – Unicidad de email: también es una regla de unicidad del sistema
38         if (await _repository.ExistsByEmailAsync(request.Email))
39         {
40             await _outputPort.HandleDuplicateEmailAsync(request.Email);
41             return;
42         }
43
44         try
45         {
46             // PASO 3 – Construcción: el dominio aplica sus invariantes (Value Objects)
47             // si algún dato viola una regla, Customer.Create lanza DomainException
48             var customer = Customer.Create(
49                 request.FirstName,
50                 request.LastName,
51                 request.Document,
52                 request.Email,
53                 request.Phone,
54                 request.Address,
55                 request.BirthDate
56             );
57
58             // PASO 4 – Persistencia: el repositorio agrega la entidad
59             await _repository.AddAsync(customer);
60
61             // PASO 5 – Notificación de éxito: construye el DTO de respuesta
62             var response = new CreateCustomerResponse
63             {
64                 CustomerId = customer.Id,
65                 FirstName = customer.FirstName.Value,
66                 LastName = customer.LastName.Value
67             };
68
69             await _outputPort.HandleSuccessAsync(response);
```

```

70     }
71     catch (DomainException ex)
72     {
73         // capturamos solo DomainException — las reglas del dominio fallaron
74         // se redirige al puerto de salida — nunca se relanza hacia afuera
75         await _outputPort.HandleValidationErrorAsync(ex.Message);
76     }
77 }
78 }

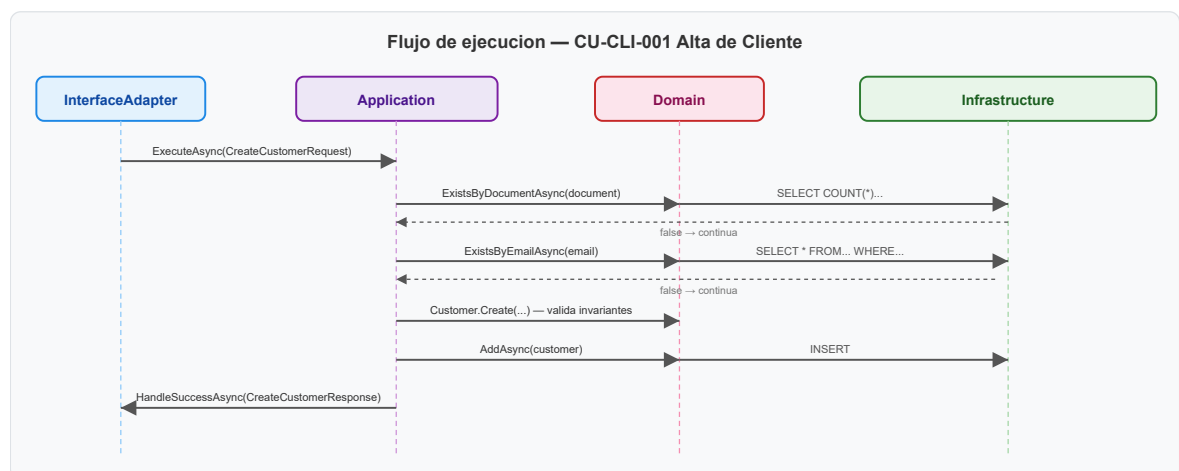
```

¿Por qué solo se captura `DomainException` y no `Exception`?

Capturar `Exception` genérico oculta errores inesperados — un fallo de red, un error de base de datos o un bug en el código quedarían silenciados. Solo se captura lo que se espera y se sabe manejar. Los errores imprevistos deben propagarse para que el middleware de la aplicación los registre y los exponga correctamente.

Explicación paso a paso

La implementación se realiza en cuatro etapas ordenadas. Cada etapa debe compilar correctamente antes de pasar a la siguiente, porque las clases posteriores dependen de las anteriores.



Paso 1 — Declarar `ICustomerRepository` en Domain

Antes de crear la capa Application, Domain necesita su contrato de persistencia. Este archivo debe existir primero porque el UseCase lo referencia. La interfaz retorna `bool` en las verificaciones de unicidad — el UseCase no necesita obtener la entidad completa, solo saber si existe.

Paso 2 — Crear los DTOs

Los DTOs deben existir antes que los puertos porque los puertos los referencian en sus firmas de métodos. Si se crean los puertos primero, el compilador reportará errores por tipos no encontrados. Los DTOs son simples contenedores de datos: sin constructores personalizados, sin lógica, sin herencia. Todos sus campos son tipos primitivos de C# — nunca Value Objects del dominio.

Paso 3 — Definir los puertos

Los puertos formalizan la comunicación entre capas. El InputPort lo implementa el UseCase. El OutputPort lo implementa la capa InterfaceAdapter. Gracias a esta inversión de dependencias, Application no necesita saber nada sobre cómo se muestra el resultado — ni si es Blazor, una API REST o una consola.

Paso 4 — Implementar CreateCustomerUseCase

El caso de uso es el punto de orquestación. Recibe el request, verifica unicidad de documento y email, delega la creación al dominio, persiste y notifica. No contiene validaciones de formato — eso lo hacen los Value Objects. Captura solo `DomainException` y la redirige al puerto de salida. Las excepciones inesperadas se dejan propagar.

Resumen de ubicaciones

Clase / Interfaz	Proyecto	Carpeta	Tipo
<code>ICustomerRepository</code>	CommerX.Domain	Customers/Repositories/	interface — contrato de persistencia
<code>CreateCustomerRequest</code>	CommerX.Application	Customers/DTOs/	class — DTO de entrada
<code>CreateCustomerResponse</code>	CommerX.Application	Customers/DTOs/	class — DTO de salida
<code>ICreateCustomerInputPort</code>	CommerX.Application	Customers/Ports/	interface — puerto de entrada

Clase / Interfaz	Proyecto	Carpeta	Tipo
<code>ICreateCustomerOutputPort</code>	CommerX.Application	Customers/Ports/	interface — puerto de salida (4 métodos)
<code>CreateCustomerUseCase</code>	CommerX.Application	Customers/UseCases/	sealed class — caso de uso
<code>DependencyContainer</code> — <code>AddCustomerUseCases()</code>	CommerX.Application	raíz del proyecto	static class — registro de dependencias

Buenas prácticas y errores comunes

DTOs — tipos primitivos vs. tipos del dominio

ERROR COMUN

El DTO importa Value Objects del dominio — Application queda acoplada a Domain en ambas direcciones.

```
1 // MAL: el DTO depende de tipos del dominio
2 public class CreateCustomerRequest
3 {
4     // Email es un Value Object de CommerX.Domain
5     public required Email Email { get; init; }
6     // BirthDate es un Value Object de CommerX.Domain
7     public required BirthDate BirthDate { get; init; }
8 }
```

BUENA PRACTICA

El DTO usa solo tipos primitivos de C#. El UseCase construye los Value Objects al llamar a `Customer.Create()`.

```
1 // BIEN: el DTO no conoce nada del dominio
2 public class CreateCustomerRequest
3 {
4     // string primitivo - sin dependencia del dominio
5     public required string Email { get; init; }
6     // DateOnly primitivo - no el Value Object BirthDate del dominio
7     public required DateOnly BirthDate { get; init; }
8 }
```

Validaciones — UseCase vs. Value Objects**ERROR COMUN**

El UseCase valida el formato del email o la mayoría de edad — esa lógica pertenece al dominio, no a la capa Application.

```
1 // MAL: lógica de dominio filtrada en el UseCase
2 if (!request.Email.Contains("@"))
3 {
4     await _outputPort.HandleValidationErrorAsync("Email invalido");
5     return;
6 }
```

BUENA PRACTICA

El UseCase delega al dominio. Si el email es inválido, `Customer.Create()` lanza `DomainException` que el `catch` redirige al OutputPort.

```
1 // BIEN: el dominio valida al construir la entidad
2 var customer = Customer.Create(...);
3 // si el email es inválido, el Value Object Email lanza
4 // InvalidEmailException - el catch lo redirige al OutputPort
```

Manejo de excepciones — específico vs. genérico

ERROR COMUN

Capturar `Exception` genérico oculta errores inesperados — bugs, fallos de red y errores de base de datos quedan silenciados.

```
1 // MAL: catch genérico oculta todo tipo de fallo
2 catch (Exception ex)
3 {
4     _outputPort.HandleValidationErrorAsync(ex.Message);
5 }
```

BUENA PRACTICA

Capturar solo `DomainException` — los errores inesperados se propagan para que el middleware los registre.

```
1 // BIEN: solo capturamos lo que sabemos manejar
2 catch (DomainException ex)
3 {
4     await _outputPort.HandleValidationErrorAsync(ex.Message);
5 }
6 // Exception inesperada → se propaga → el middleware la registra
```

Actividad

Implementá de forma completa el caso de uso **CU-CLI-001 — Alta de Cliente** en el proyecto **CommerX**, siguiendo el orden de las partes. Compilar el proyecto después de cada parte antes de continuar.

Parte A — Repositorio en Domain

1. Crear la carpeta `Customers/Repositories/` dentro de `CommerX.Domain`.
2. Declarar el namespace: `CommerX.Domain.Customers.Repositories`.
3. Agregar: `using CommerX.Domain.Customers.Entities;`
4. Declarar la interfaz `ICustomerRepository` con los tres métodos: `ExistsByIdAsync`, `ExistsByEmailAsync` y `AddAsync`.
5. Compilar `CommerX.Domain` — debe compilar sin errores.

Antes de continuar, escribí como comentario al inicio del archivo por qué esta interfaz vive en Domain y no en Application. Debe mencionar la dirección de las dependencias y qué ocurriría si se colocara en Application.

Parte B — Proyecto Application y DTOs

Si el proyecto `CommerX.Application` no existe aún, crearlo: clic derecho en la solución → **Agregar** → **Nuevo proyecto** → **Biblioteca de clases (.NET 9)** → nombre: `CommerX.Application`. Luego agregar la referencia a Domain en el `.csproj`:

```
1 <!-- CommerX.Application.csproj - referencia unidireccional a Domain -->
2 <ItemGroup>
3   <ProjectReference Include="..\CommerX.Domain\CommerX.Domain.csproj" />
4 </ItemGroup>
```

B.1 — `CreateCustomerRequest.cs`

1. Crear carpeta `Customers/DTOs/` en `CommerX.Application`.
2. Namespace: `CommerX.Application.Customers.DTOs`.
3. Declarar `public class CreateCustomerRequest` con 7 propiedades `required + init`: `FirstName`, `LastName`, `Document`, `Email`, `Phone`, `Address` (todas `string`) y `BirthDate` (`DateOnly`).
4. Verificar que no hay Value Objects del dominio entre los tipos.

B.2 — `CreateCustomerResponse.cs`

1. Mismo namespace.
2. Declarar `public class CreateCustomerResponse` con 3 propiedades `required + init`: `CustomerId` (`Guid`), `FirstName` y `LastName` (`string`).
3. Compilar antes de continuar.

Parte C — Puertos

Crear carpeta `Customers/Ports/` en `CommerX.Application`.

C.1 — `ICreateCustomerInputPort.cs`

1. Namespace: `CommerX.Application.Customers.Ports`.
2. Agregar `using CommerX.Application.Customers.DTOs;`
3. Declarar la interfaz con un único método: `Task ExecuteAsync(CreateCustomerRequest request);`

C.2 — ICreateCustomerOutputPort.cs

1. Mismo namespace y using de DTOs.
2. Declarar la interfaz con los cuatro métodos de notificación usando `Task` (no `void`): `HandleSuccessAsync`, `HandleDuplicateDocumentAsync`, `HandleDuplicateEmailAsync` y `HandleValidationErrorAsync`.
3. Compilar antes de continuar.

Parte D — Caso de uso

1. Crear carpeta `Customers/UseCases/` en `CommerX.Application`.
2. Namespace: `CommerX.Application.Customers.UseCases`.
3. Agregar todos los `using` necesarios: `DTOs`, `Ports`, `Domain.Common.Exceptions`, `Domain.Customers.Entities`, `Domain.Customers.Repositories`.
4. Declarar: `public sealed class CreateCustomerUseCase : ICreateCustomerInputPort`
5. Inyectar `ICustomerRepository` e `ICreateCustomerOutputPort` por constructor.
6. Implementar `ExecuteAsync` siguiendo el flujo del ejemplo:
 - a. Verificar unicidad de documento. Si existe, llamar a `HandleDuplicateDocumentAsync` y retornar.
 - b. Verificar unicidad de email. Si existe, llamar a `HandleDuplicateEmailAsync` y retornar.
 - c. Crear la entidad con `Customer.Create(...)` dentro de un `try`.
 - d. Persistir con `AddAsync`.
 - e. Construir `CreateCustomerResponse` y llamar a `HandleSuccessAsync`.
 - f. En el `catch` capturar solo `DomainException` y llamar a `HandleValidationErrorAsync`.
7. Compilar la solución completa — debe compilar sin errores ni warnings.

Parte E — Registro de dependencias

Antes de continuar: leer el anexo.

Esta parte introduce el registro de dependencias en el contenedor de .NET. Para comprender *por qué* existe cada elemento — IoC, `IServiceCollection`, ciclos de vida y métodos de extensión — es necesario leer primero el **Anexo 04 — Inyección de Dependencias** (`anexo-04-inyeccion-dependencias.html`) antes de ejecutar los pasos siguientes.

Con el caso de uso implementado, el sistema necesita saber qué clase concreta debe instanciar cuando alguien pida un `ICreateCustomerInputPort`. Esa información se registra en el **contenedor de dependencias** de .NET, y por convención cada capa expone un método de extensión propio para registrar sus implementaciones.

E.1 — Agregar el paquete NuGet al proyecto

Abrir `CommerX.Application.csproj` y agregar la referencia al paquete de abstracciones del contenedor. Se usa el paquete de *abstracciones* (no el completo) para que `Application` no dependa de la implementación concreta del contenedor:

```
1 <!-- CommerX.Application.csproj -->
2 <ItemGroup>
3   <PackageReference Include="Microsoft.Extensions.DependencyInjection.Abstractions"
4                       Version="9.0.0" />
5 </ItemGroup>
```

E.2 — Crear `DependencyContainer.cs` en la raíz de `Application`

1. Crear el archivo `DependencyContainer.cs` directamente en la raíz del proyecto `CommerX.Application` (no dentro de ninguna subcarpeta).
2. Declarar el namespace como `Microsoft.Extensions.DependencyInjection` — el mismo namespace del paquete. Esto permite que `Program.cs` llame al método de extensión sin necesidad de agregar un `using` adicional.
3. Implementar el método de extensión `AddCustomerUseCases`.

```
1 // CommerX.Application/DependencyContainer.cs
2 // namespace truco: al declararlo igual que el paquete,
3 // Program.cs no necesita un using extra para ver AddCustomerUseCases()
4 namespace Microsoft.Extensions.DependencyInjection;
5
6 using CommerX.Application.Customers.Ports;
7 using CommerX.Application.Customers.UseCases;
8
9 // static: clase de utilidad – no se instancia
10 public static class DependencyContainer
11 {
12     // método de extensión sobre IServiceCollection
13     // retorna IServiceCollection para permitir el encadenamiento fluente
14     public static IServiceCollection AddCustomerUseCases(
15         this IServiceCollection services)
16     {
17         // Scoped: una instancia por request HTTP – mismo ciclo de vida que DbContext
18         // cuando alguien pida ICreateCustomerInputPort, el contenedor entregará
19         // una instancia de CreateCustomerUseCase con sus dependencias ya inyectadas
20         services.AddScoped<ICreateCustomerInputPort, CreateCustomerUseCase>();
21     }
22 }
```

```

21
22     // retorna services para habilitar el encadenamiento en Program.cs
23     return services;
24 }
25 }

```

¿Por qué `AddScoped` y no `AddTransient` ?

El `UseCase` recibe un `ICustomerRepository` que internamente usa un `DbContext`. El `DbContext` de Entity Framework Core se registra como *Scoped* — una instancia por request. Si el `UseCase` fuera *Transient*, el contenedor crearía una nueva instancia en cada inyección pero con el mismo `DbContext` del request, generando inconsistencias. Usando *Scoped*, `UseCase` y `DbContext` comparten el mismo ciclo de vida dentro del mismo request.

E.3 — Verificar el encadenamiento en `Program.cs`

Una vez que `Infrastructure` también exponga su `AddCustomerInfrastructure()` (próximo módulo), `Program.cs` encadenará los registros de todas las capas en una sola llamada:

```

1 // Program.cs – encadenamiento fluente de registros por capa
2 // no requiere using adicional – el namespace truco lo resuelve
3 builder.Services
4     .AddCustomerUseCases() // CommerX.Application
5     .AddCustomerInfrastructure() // CommerX.Infrastructure – próximo módulo
6     .AddCustomerPresenter(); // CommerX.InterfaceAdapter – módulo 07

```

1. Compilar `CommerX.Application` — debe compilar sin errores.
2. Verificar que el namespace del archivo es `Microsoft.Extensions.DependencyInjection` (no `CommerX.Application`).

Parte F — Checklist de entrega

Archivo	Ubicación	Verificación
<code>ICustomerRepository.cs</code>	<code>CommerX.Domain / Customers/Repositories/</code>	Compila sin errores
<code>CreateCustomerRequest.cs</code>	<code>CommerX.Application / Customers/DTOs/</code>	Solo tipos primitivos
<code>CreateCustomerResponse.cs</code>	<code>CommerX.Application / Customers/DTOs/</code>	Solo tipos primitivos

Archivo	Ubicación	Verificación
<code>ICreateCustomerInputPort.cs</code>	CommerX.Application / Customers/Ports/	Un método Task
<code>ICreateCustomerOutputPort.cs</code>	CommerX.Application / Customers/Ports/	Cuatro métodos Task
<code>CreateCustomerUseCase.cs</code>	CommerX.Application / Customers/UseCases/	Sealed, catch DomainException
<code>DependencyContainer.cs</code>	CommerX.Application / raíz	Namespace truco, AddScoped

Parte G — Preguntas de reflexión

1. ¿Por qué `ICustomerRepository` se declara en Domain y no en Application?
2. ¿Qué ocurriría si el DTO `CreateCustomerRequest` usara el tipo `Email` del dominio en lugar de `string`?
3. ¿Por qué el UseCase no valida el formato del email directamente?
4. ¿Cuál es la diferencia entre capturar `DomainException` y capturar `Exception`? ¿Qué riesgo introduce el segundo?
5. ¿Qué ventaja ofrece el patrón OutputPort frente a que el UseCase retorne directamente un objeto de resultado?
6. ¿Por qué se declara el namespace de `DependencyContainer.cs` como `Microsoft.Extensions.DependencyInjection` en lugar de `CommerX.Application`?